COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	July 21, 2026	**DATE SUBMITTED:**	July 28, 2026
NAME:	Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 **Creating a Simply Linked List**

1. PROCEDURE OUTPUT

What procedures did you perform?

UNDER THE main.cpp

main.cpp simply_LL.h

main.cpp > ...

```cpp
#include <iostream>
#include "simply_LL.h"

int main(){

    // Creating an empty list
    SimplyList<char>* head = new SimplyList<char>;
    SimplyList<char>* second = new SimplyList<char>;
    SimplyList<char>* third = new SimplyList<char>;
    SimplyList<char>* fourth = new SimplyList<char>;
    SimplyList<char>* fifth = new SimplyList<char>;

    // Assign the data:
    head->data = 'F';
    head->next = second;

    second->data = 'E';
    second->next = third;

    third->data = 'N';
    third->next = fourth;

    fourth->data = 'Z';
    fourth->next = nullptr;
```

main.cpp simply_LL.h

main.cpp > ...

```cpp
25
26
27      std::cout<<"original list:"<<std::endl;
28      listTraversal(head);
29
30      std::cout<<"inserting a new node at the head: \n";
31      sllinsertHead('Q',&head);
32      listTraversal(head);
33
34      std::cout<<"inserting a new node at the third: \n";
35      sllGeneralInsert('X', head->next->next);
36      listTraversal(head);
37
38      std::cout<<"inserting a new node at the END: \n";
39      sllinsertEnd('U', &head);
40      listTraversal(head);
41
42      std::cout<<"Deleting a node: \n";
43      sllDeleteNode('X', &head);
44      listTraversal(head);
45
46      std::cout<<"Deleting the head node: \n";
47      sllDeleteNode('Q', &head);
48      listTraversal(head);
49
50      std::cout<<"Deleting all the nodes: \n";
51      sllDeleteList(&head);
52      listTraversal(head);
53
54
55
56      return 0;
57  }
```

UNDER THE simply_LL.h

simply_LL.h

simply_LL.h > sllGeneralInsert<T>(T, SimplyList<T>*)

```
1   #ifndef SIMPLY_LL_H
2   #define SIMPLY_LL_H
3
4   #include <iostream>
5
6   template <typename T>
7   class SimplyList{
8       public:
9           T data;
10          SimplyList<T> *next = nullptr;
11  };
12
13  //Traversal
14  template <typename T>
15  void listTraversal(SimplyList<T>* head){
16      if (head==nullptr){
17          std::cout<<"List is empty!"<<std::endl;
18          return;
19      }
20
21      while(head != nullptr){
22          std::cout<<head->data;
23          if (head->next != nullptr){
24              std::cout<<"->";
```

```
24              std::cout<<"->";
25          }
26
27          head = head->next;
28      }
29      std::cout<<std::endl;
30  }
31
32  /*
33  1. Allocate memory for the new code
34  2. Put our data into the new node
35  3. Set Next of the new node to point to the previous Head
36  4. Reset Head to point to the new node
37  */
38
39  template <typename T>
40  void sllinsertHead(T newData, SimplyList<T>** currentHead){
41      // 1. Allocate memory for the new code
42      SimplyList<T>* newNode = new SimplyList<T>;
43      // 2. Put our data into the new node
```

```
44      newNode->data = newData;
45      // 3. Set Next of the new node to point to the previous Hea
46      newNode->next = *currentHead;
47      // 4. Reset Head to point to the new node
48      *currentHead = newNode;
49
50  }
51  /*
52  2. If null, print
53
54  */
55  template <typename T>
56  void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
57      // 2. If null, print "Previous node cannot be null."
58      if(prevNode == nullptr){
59          std::cout<<"Previous Node cannot be Null" <<std::endl;
60          return;
61      }
62      //3. Allocate a new node
63      SimplyList<T>* newNode = new SimplyList<T>;
```

```
64
65      //4. Store data in the new node
66      newNode->data = newData;
67
68      //5. Point new node to the node previous node pointing to
69      newNode->next = prevNode->next;
70
71      //6. Point previous node to the new code
72      prevNode->next = newNode;
73  }
74
75  template<typename T>
76  void sllinsertEnd(T newData, SimplyList<T>** head){
77      //1. Allocate new node
78      SimplyList<T>* newNode = new SimplyList<T>;
79      // 2. Dereference to the head node
80      SimplyList<T>* currentNode = *head;
81      //3. Store data in new node
82      newNode->data = newData;
83      // 4. Point next of new node to NULL
84      newNode->next = nullptr;
85      // 5. Traverse the list until next of the node is null
```

```
86      while(currentNode->next != nullptr){
87          currentNode = currentNode->next;
88      }
89      // 6. Point the next of the current node to the new node
90      currentNode->next = newNode;
91
92  }
93
94  template <typename T>
95  void sllDeleteNode(T findData, SimplyList<T>** head){
96      // check if there is no list to be deleted.
97      if(*head ==nullptr) return;
98
99      SimplyList<T>* currNode =*head;
100     SimplyList<T>* prevNode = nullptr;
101
102     // searching loop
103     while(currNode != nullptr && currNode->data != findData){
104         prevNode = currNode;
105         currNode = currNode->next;
106     }
107
108     // data was not found:
109     if (currNode == nullptr) return;
110
111     // what if the node to be deleted is the head node
112     if(prevNode == nullptr){
113         *head = currNode->next;
114     }else{
115         prevNode->next = currNode->next;
116     }
117
118     // we have to delete the node?
119     delete currNode;
120
121
122 }
123
124 template <typename T>
125 void sllDeleteList(SimplyList<T>** head){
126     SimplyList<T>* current = *head;
127
128     while(current != nullptr){
```

```
127
128     while(current != nullptr){
129         SimplyList<T>* temp = current;
130         current = current->next;
131         delete temp;
132     }
133     *head = nullptr;
134 }
135
136
137 /*
138 1. Find the previous node of the node to be deleted.
139 2. Change the next of previous node.
140 3. Free memory for the to be deleted.
141 */
142 #endif
```

OUTPUTS:

```
[Running] cd "c:\Users\TIPQC\Documents\CPE_REYES\" && g++ main.cpp -o main &
F->E->N->Zinserting a new node at the head:
Q->F->E->N->Z
[Done] exited with code=0 in 0.4 seconds
```

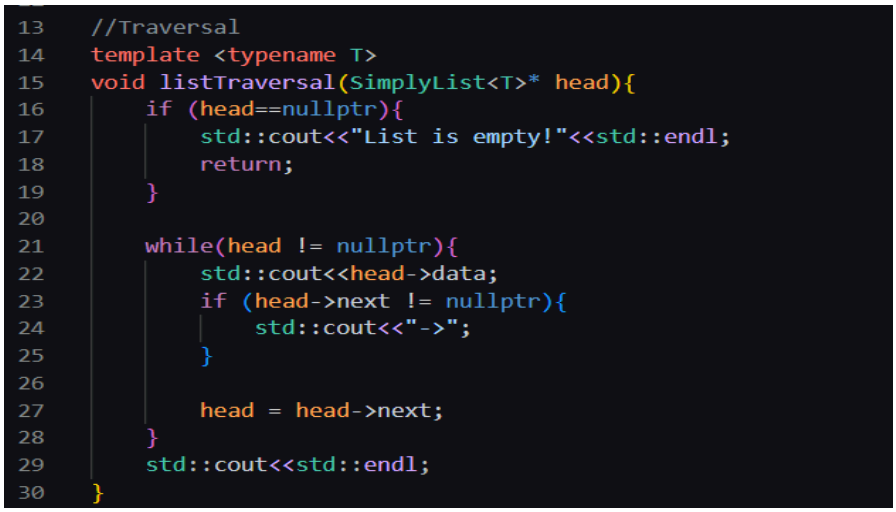
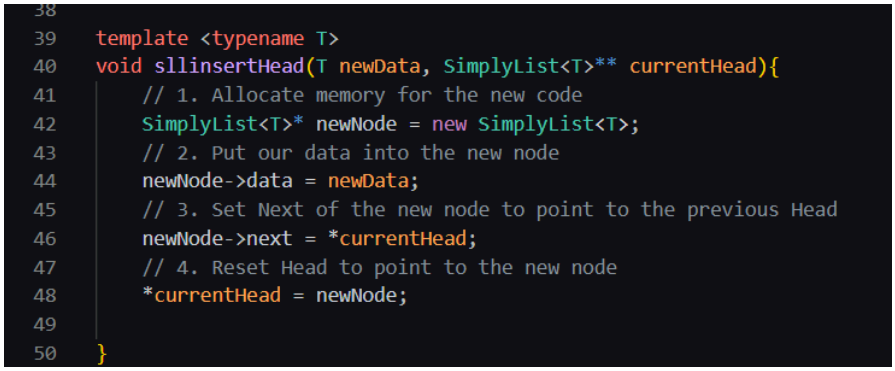
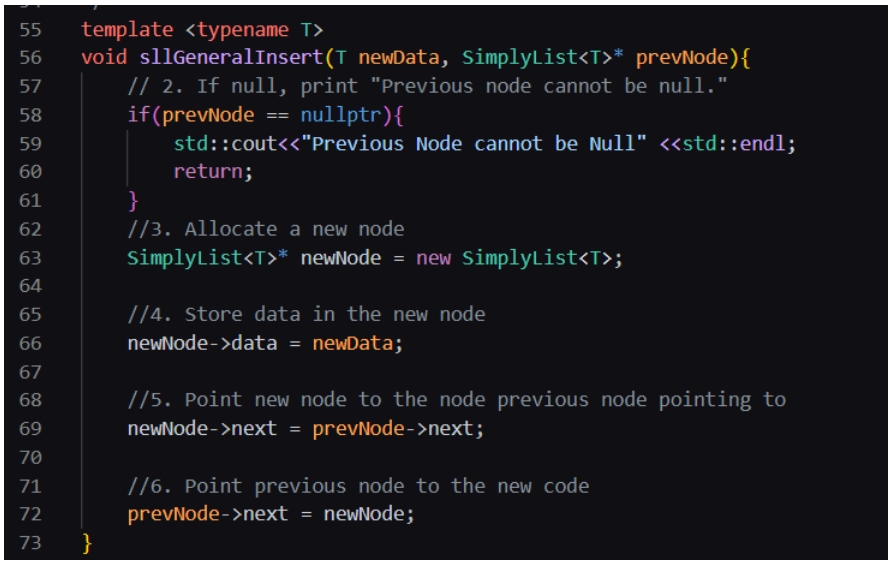
```
[Running] cd "c:\Users\TIPQC\Documents\CPE_REYES\" && g++ main.cpp -o main
original list:
F->E->N->Z
inserting a new node at the head:
Q->F->E->N->Z
inserting a new node at the head:
Q->F->E->X->N->Z
```

```
[Running] cd "c:\Users\TIPQC\Documents\CPE_REYES\" && g++ main.cpp -o main && "c:\Users\T
original list:
F->E->N->Z
inserting a new node at the head:
Q->F->E->N->Z
inserting a new node at the third:
Q->F->E->X->N->Z
inserting a new node at the END:
Q->F->E->X->N->Z->U
```

```
[Running] cd "c:\Users\TIPQC\Documents\CPE_REYES\" && g++ main.cpp -o main && "
original list:
F->E->N->Z
inserting a new node at the head:
Q->F->E->N->Z
inserting a new node at the third:
Q->F->E->X->N->Z
inserting a new node at the END:
Q->F->E->X->N->Z->U
Deleting a node:
Q->F->E->N->Z->U
```

```
[Running] cd "c:\Users\TIPQC\Documents\CPE_REYES\" && g++ main.cpp -o main && "c:\Users\T
original list:
F->E->N->Z
inserting a new node at the head:
Q->F->E->N->Z
inserting a new node at the third:
Q->F->E->X->N->Z
inserting a new node at the END:
Q->F->E->X->N->Z->U
Deleting a node:
Q->F->E->N->Z->U
Deleting the head node:
F->E->N->Z->U
```

```
[Running] cd "c:\Users\TIPQC\Documents\CPE_REYES\" && g++ main.cpp -o main && "c:\Users\T
original list:
F->E->N->Z
inserting a new node at the head:
Q->F->E->N->Z
inserting a new node at the third:
Q->F->E->X->N->Z
inserting a new node at the END:
Q->F->E->X->N->Z->U
Deleting a node:
Q->F->E->N->Z->U
Deleting the head node:
F->E->N->Z->U
Deleting all the nodes:
List is empty!
```

Operation	Screenshot
Transversal	
Insertion at head	
Insertion at any part of the list	

```cpp
//Traversal
template <typename T>
void listTraversal(SimplyList<T>* head){
    if (head==nullptr){
        std::cout<<"List is empty!"<<std::endl;
        return;
    }

    while(head != nullptr){
        std::cout<<head->data;
        if (head->next != nullptr){
            std::cout<<"->";
        }

        head = head->next;
    }
    std::cout<<std::endl;
}
```

```cpp
template <typename T>
void sllinsertHead(T newData, SimplyList<T>** currentHead){
    // 1. Allocate memory for the new code
    SimplyList<T>* newNode = new SimplyList<T>;
    // 2. Put our data into the new node
    newNode->data = newData;
    // 3. Set Next of the new node to point to the previous Head
    newNode->next = *currentHead;
    // 4. Reset Head to point to the new node
    *currentHead = newNode;

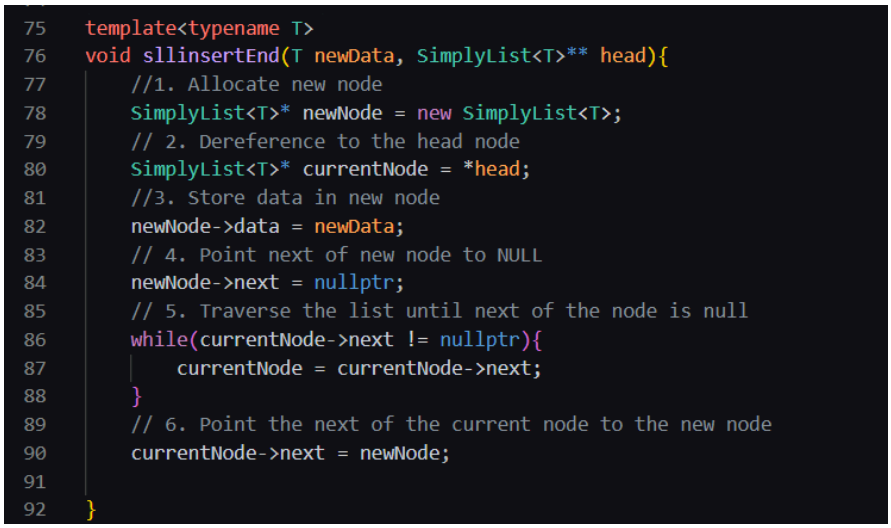
}
```

```cpp
template <typename T>
void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
    // 2. If null, print "Previous node cannot be null."
    if(prevNode == nullptr){
        std::cout<<"Previous Node cannot be Null" <<std::endl;
        return;
    }
    //3. Allocate a new node
    SimplyList<T>* newNode = new SimplyList<T>;

    //4. Store data in the new node
    newNode->data = newData;

    //5. Point new node to the node previous node pointing to
    newNode->next = prevNode->next;

    //6. Point previous node to the new code
    prevNode->next = newNode;
}
```

Insertion at the end	
Deletion of a node	

```
75  template<typename T>
76  void sllinsertEnd(T newData, SimplyList<T>** head){
77      //1. Allocate new node
78      SimplyList<T>* newNode = new SimplyList<T>;
79      // 2. Dereference to the head node
80      SimplyList<T>* currentNode = *head;
81      //3. Store data in new node
82      newNode->data = newData;
83      // 4. Point next of new node to NULL
84      newNode->next = nullptr;
85      // 5. Traverse the list until next of the node is null
86      while(currentNode->next != nullptr){
87          currentNode = currentNode->next;
88      }
89      // 6. Point the next of the current node to the new node
90      currentNode->next = newNode;
91
92  }
```

```
94  template <typename T>
95  void sllDeleteNode(T findData, SimplyList<T>** head){
96      // check if there is no list to be deleted.
97      if(*head ==nullptr) return;
98
99      SimplyList<T>* currNode =*head;
100     SimplyList<T>* prevNode = nullptr;
101
102     // searching loop
103     while(currNode != nullptr && currNode->data != findData){
104         prevNode = currNode;
105         currNode = currNode->next;
106     }
107
108     // data was not found:
109     if (currNode == nullptr) return;
110
111     // what if the node to be deleted is the head node
112     if(prevNode == nullptr){
113         *head = currNode->next;
114     }else{
115         prevNode->next = currNode->next;
116     }
117
118     // we have to delete the node?
119     delete currNode;
120
121
122 }
```

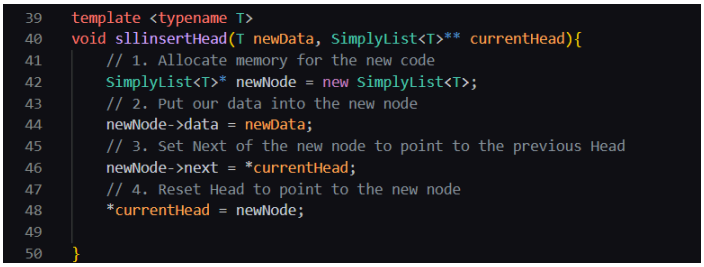
Table 3-2. Code for the List Operations

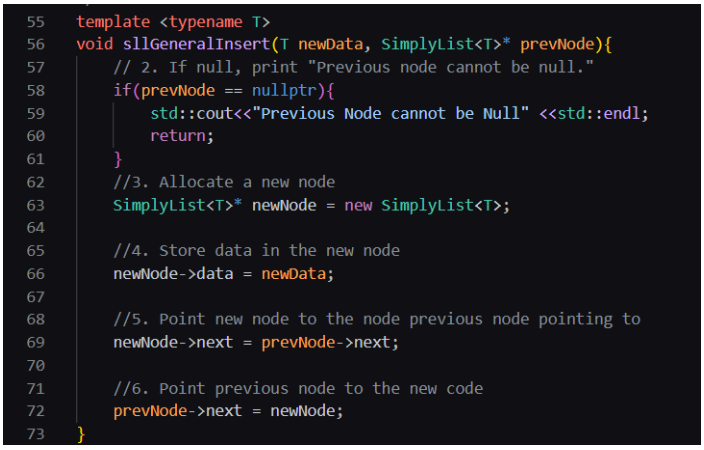
a.	**Source Code**	listTraversal(head);
	Console	F->E->N->Z
b.	**Source Code**	sllinsertHead('Q', &head);
	Console	Q->F->E->N->Z
c.	**Source Code**	sllGeneralInsert('X', head->next->next);
	Console	Q->F->E->X->N->Z
d.	**Source Code**	sllinsertEnd('U', &head);
	Console	Q->F->E->X->N->Z->U
e.	**Source Code**	sllDeleteNode('X', &head);
	Console	Q->F->E->N->Z->U
f.	**Source Code**	sllDeleteNode('Q', &head); sllDeleteList(&head);
	Console	F->E->N->Z->U List is empty!

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots(s)	Analysis
 5 6 template <typename T> 7 class SimplyList{ 8 public: 9 T data; 10 SimplyList<T> *next = nullptr; 11 };	**Structure Analysis:** Defines a generic singly linked list node using C++ templates. Each node holds data of type T and a single pointer (next) pointing to the following node, allowing unidirectional forward traversal only.


13 //Traversal
14 template <typename T>
15 void listTraversal(SimplyList<T>* head){
16 if (head==nullptr){
17 std::cout<<"List is empty!"<<std::endl;
18 return;
19 }
20
21 while(head != nullptr){
22 std::cout<<head->data;
23 if (head->next != nullptr){
24 std::cout<<"->";
25 }
26
27 head = head->next;
28 }
29 std::cout<<std::endl;
30 }
31
	Traversal Analysis: Iterates through the list starting from head using a while loop until head != nullptr. It prints each node's data value separated by -> without modifying any links in memory. Time complexity is O(n).

 39 template <typename T> 40 void sllinsertHead(T newData, SimplyList<T>** currentHead){ 41 // 1. Allocate memory for the new code 42 SimplyList<T>* newNode = new SimplyList<T>; 43 // 2. Put our data into the new node 44 newNode->data = newData; 45 // 3. Set Next of the new node to point to the previous Head 46 newNode->next = *currentHead; 47 // 4. Reset Head to point to the new node 48 *currentHead = newNode; 49 50 }	**Head Insertion Analysis:** Allocates dynamic memory for node 'Q', sets its next pointer to the existing head, and updates head to point to the new node. This is an O(1) constant-time operation.


55 template <typename T>
56 void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
57 // 2. If null, print "Previous node cannot be null."
58 if(prevNode == nullptr){
59 std::cout<<"Previous Node cannot be Null" <<std::endl;
60 return;
61 }
62 //3. Allocate a new node
63 SimplyList<T>* newNode = new SimplyList<T>;
64
65 //4. Store data in the new node
66 newNode->data = newData;
67
68 //5. Point new node to the node previous node pointing to
69 newNode->next = prevNode->next;
70
71 //6. Point previous node to the new code
72 prevNode->next = newNode;
73 }
	Middle Insertion Analysis: Inserts node 'X' after the designated previous node (prevNode). It links newNode->next to prevNode->next, then updates prevNode->next to point to newNode, preserving the rest of the chain.

```
75  template<typename T>
76  void sllinsertEnd(T newData, SimplyList<T>** head){
77      //1. Allocate new node
78      SimplyList<T>* newNode = new SimplyList<T>;
79      // 2. Dereference to the head node
80      SimplyList<T>* currentNode = *head;
81      //3. Store data in new node
82      newNode->data = newData;
83      // 4. Point next of new node to NULL
84      newNode->next = nullptr;
85      // 5. Traverse the list until next of the node is null
86      while(currentNode->next != nullptr){
87          currentNode = currentNode->next;
88      }
89      // 6. Point the next of the current node to the new node
90      currentNode->next = newNode;
91
92  }
```

End Insertion Analysis: Traverses to the final node (where next == nullptr), then updates that node's next pointer to point to newly created node 'U'. Time complexity is O(n) due to full list traversal.

```
94  template <typename T>
95  void sllDeleteNode(T findData, SimplyList<T>** head){
96      // check if there is no list to be deleted.
97      if(*head ==nullptr) return;
98
99      SimplyList<T>* currNode =*head;
100     SimplyList<T>* prevNode = nullptr;
101
102     // searching loop
103     while(currNode != nullptr && currNode->data != findData){
104         prevNode = currNode;
105         currNode = currNode->next;
106     }
107
108     // data was not found:
109     if (currNode == nullptr) return;
110
111     // what if the node to be deleted is the head node
112     if(prevNode == nullptr){
113         *head = currNode->next;
114     }else{
115         prevNode->next = currNode->next;
116     }
117
118     // we have to delete the node?
119     delete currNode;
120
121
122 }
```

Deletion Analysis: Traverses the list tracking both currNode and prevNode. Once target node 'X' is found, it bypasses it by setting prevNode->next = currNode->next, then frees memory using delete currNode.

```
124 template <typename T>
125 void sllDeleteList(SimplyList<T>** head){
126     SimplyList<T>* current = *head;
127
128     while(current != nullptr){
129         SimplyList<T>* temp = current;
130         current = current->next;
131         delete temp;
132     }
133     *head = nullptr;
134 }
```

List Cleanup Analysis: Iterates through the list using a temp pointer to delete each node sequentially, preventing memory leaks. Finally, set head to nullptr to safely reset the list.

Table 3-4. Modified Operations for Doubly Linked Lists

CONCLUSION:

Going through these exercises made me realize the importance of linked lists in defining the core concepts of Data Structures and Algorithms in creating a valuable code that efficiently handles the memory. With the use of nodes, and different guidelines that my professor has provided, I have gained an understanding of how does memory allocation works in linked list Furthermore, memory manipulation was also discussed through the code and how each building blocks of the linked lists will be filled with information provided in the main file and simply_LL.h file. With that, I learned how to create a list, insert nodes at different phases of the list, to visit each node sequentially to process data using transversal, as well as to delete all the nodes given in the code. Though, I barely grasp the topic very well but I still gained a valuable lesson of what today's discussion wanted to convey. In connection with that, I also encountered many errors on the pointers handling which is a crucial lesson for me to overlook and be conscious of my coding work per line to avoid unnecessary errors. Overall, this laboratory exercise paved the way for me to understand and apply the concepts of linked lists, nodes, and further enhance my understanding in using class to create an efficient code.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

CODE INPUT:

loveisalltheseandmore.cpp

loveisalltheseandmore.cpp > main()

```cpp
#include <iostream>
#include <string>

class SongNode
{
public:
    std::string title;
    SongNode* next;
    SongNode* prev;

    SongNode(std::string songTitle)
    {
        title = songTitle;
        next = nullptr;
        prev = nullptr;
    }
};

class Playlist
{
private:
    SongNode* head;
    SongNode* current;
```

loveisalltheseandmore.cpp

loveisalltheseandmore.cpp > main()

```cpp
class Playlist

public:
    Playlist()
    {
        head = nullptr;
        current = nullptr;
    }

    // Add song to playlist
    void insertSong(std::string title)
    {
        SongNode* newNode = new SongNode(title);

        if (head == nullptr)
        {
            head = newNode;
            head->next = head;
            head->prev = head;
            current = head;
        }
        else
        {
            SongNode* last = head->prev;

            last->next = newNode;
            newNode->prev = last;

            newNode->next = head;
            head->prev = newNode;
        }
```

loveisalltheseandmore.cpp

loveisalltheseandmore.cpp > main()

```
19  class Playlist
33      void insertSong(std::string title)
44          else
54
55          std::cout << "\"" << title << "\" added to playlist.\n";
56      }
57
58      // Remove a song
59      void removeSong(std::string title)
60      {
61          if (head == nullptr)
62          {
63              std::cout << "Playlist is empty.\n";
64              return;
65          }
66
67          SongNode* temp = head;
68
69          do
70          {
71              if (temp->title == title)
72              {
73                  if (temp->next == temp)
74                  {
75                      delete temp;
76                      head = nullptr;
77                      current = nullptr;
78                  }
79                  else
80                  {
81                      temp->prev->next = temp->next;
82                      temp->next->prev = temp->prev;
```

loveisalltheseandmore.cpp

loveisalltheseandmore.cpp > main()

```
19  class Playlist
59      void removeSong(std::string title)
69          do
71              if (temp->title == title)
79                  else
83
84                      if (temp == head)
85                          head = temp->next;
86
87                      if (temp == current)
88                          current = temp->next;
89
90                      delete temp;
91                  }
92
93                  std::cout << "\"" << title << "\" removed.\n";
94                  return;
95              }
96
97              temp = temp->next;
98
99          } while (temp != head);
100
101         std::cout << "Song not found.\n";
102     }
103
104     // Next song
105     void nextSong()
106     {
107         if (current == nullptr)
108         {
109             std::cout << "Playlist is empty.\n";
110             return;
```

loveisalltheseandmore.cpp

loveisalltheseandmore.cpp > main()

```
19   class Playlist
105      void nextSong()
107          if (current == nullptr)
111          }
112
113          current = current->next;
114          std::cout << "Now Playing: " << current->title << std::endl;
115      }
116
117      // Previous song
118      void previousSong()
119      {
120          if (current == nullptr)
121          {
122              std::cout << "Playlist is empty.\n";
123              return;
124          }
125
126          current = current->prev;
127          std::cout << "Now Playing: " << current->title << std::endl;
128      }
129
130      // Display current song
131      void currentSong()
132      {
133          if (current == nullptr)
134          {
135              std::cout << "Playlist is empty.\n";
136          }
137          else
138          {
139              std::cout << "Current Song: " << current->title << std::endl;
140          }
```

loveisalltheseandmore.cpp

loveisalltheseandmore.cpp > main()

```
19    class Playlist
131       void currentSong()
141       }
142
143       // Play all songs once
144       void playAllSongs()
145       {
146           if (head == nullptr)
147           {
148               std::cout << "Playlist is empty.\n";
149               return;
150           }
151
152           SongNode* temp = head;
153
154           std::cout << "\n--- Playlist ---\n";
155
156           do
157           {
158               std::cout << temp->title << std::endl;
159               temp = temp->next;
160           }
161           while (temp != head);
162
163           std::cout << "----------------\n";
164       }
165   };
166
```

loveisalltheseandmore.cpp

loveisalltheseandmore.cpp > main()

```cpp
int main()
{
    Playlist playlist;

    int choice;
    std::string song;

    do
    {
        std::cout << "\n===== SONG PLAYLIST =====\n";
        std::cout << "1. Add Song\n";
        std::cout << "2. Remove Song\n";
        std::cout << "3. Next Song\n";
        std::cout << "4. Previous Song\n";
        std::cout << "5. Current Song\n";
        std::cout << "6. Play All Songs\n";
        std::cout << "7. Exit\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;

        std::cin.ignore();

        switch(choice)
        {
        case 1:
            std::cout << "Enter song title: ";
            std::getline(std::cin, song);
            playlist.insertSong(song);
            break;

```

loveisalltheseandmore.cpp

loveisalltheseandmore.cpp > main()

```
197         case 2:
198             std::cout << "Enter song title to remove: ";
199             std::getline(std::cin, song);
200             playlist.removeSong(song);
201             break;
202 
203         case 3:
204             playlist.nextSong();
205             break;
206 
207         case 4:
208             playlist.previousSong();
209             break;
210 
211         case 5:
212             playlist.currentSong();
213             break;
214 
215         case 6:
216             playlist.playAllSongs();
217             break;
218 
219         case 7:
220             std::cout << "Exiting Playlist...\n";
221             break;
222 
223         default:
224             std::cout << "Invalid choice.\n";
225         }
226 
227     } while(choice != 7);
228 
229     return 0;
```

```
229     return 0;
230 }
```

CODE OUTPUT:

```
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\REYES_HOA3_
\extensions\ms-vscode.cpptools-1.32.2-win32-x64\debugAdapters
MIEngine-In-q5g14clb.4nn' '--stdout=Microsoft-MIEngine-Out-tw
vau25.50e' '--pid=Microsoft-MIEngine-Pid-srcxioch.qjx' '--dbg
i'

===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 1
Enter song title: Be With You - Ridleys
"Be With You - Ridleys" added to playlist.

===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 1
Enter song title: Milkyway - Ridleys
"Milkyway - Ridleys" added to playlist.

===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 1
Enter song title: Rorschach Blots - Ridleys
"Rorschach Blots - Ridleys" added to playlist.
```

```
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\REYES_HOA3_S
\extensions\ms-vscode.cpptools-1.32.2-win32-x64\debugAdapters\
MIEngine-In-q5g14clb.4nn' '--stdout=Microsoft-MIEngine-Out-twr
vau25.50e' '--pid=Microsoft-MIEngine-Pid-srcxioch.qjx' '--dbgE
i'
===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 1
Enter song title: Someday - Ridleys
"Someday - Ridleys" added to playlist.

===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 1
Enter song title: Someday - Ridleys
"Someday - Ridleys" added to playlist.

===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 1
Enter song title: Love Is - Ridleys
"Love Is - Ridleys" added to playlist.
```

```
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\REYES_HO
\extensions\ms-vscode.cpptools-1.32.2-win32-x64\debugAdapt
MIEngine-In-q5g14clb.4nn' '--stdout=Microsoft-MIEngine-Out
vau25.50e' '--pid=Microsoft-MIEngine-Pid-srcxioch.qjx' '--
i'
===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 1
Enter song title: Euphoria - Ridleys
"Euphoria - Ridleys" added to playlist.

===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 6

--- Playlist ---
Be With You - Ridleys
Milkyway - Ridleys
Rorschach Blots - Ridleys
Someday - Ridleys
Someday - Ridleys
Love Is - Ridleys
Euphoria - Ridleys
---------------
```

```
===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 3
Now Playing: Milkyway - Ridleys

===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice:
```

```
===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 2
Enter song title to remove: Someday - Ridleys
"Someday - Ridleys" removed.
```

```
===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 6

--- Playlist ---
Be With You - Ridleys
Milkyway - Ridleys
Rorschach Blots - Ridleys
Someday - Ridleys
Love Is - Ridleys
Euphoria - Ridleys
---------------
```

```
===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 4
Now Playing: Be With You - Ridleys
```

```
===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 5
Current Song: Be With You - Ridleys
```

```
===== SONG PLAYLIST =====
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Current Song
6. Play All Songs
7. Exit
Enter choice: 7
Exiting Playlist...
PS C:\Users\Angel Elayza Reyes\OneDrive\Documents\REYES HOA3 SUPP SONG>
```

CODE ANALYSIS:

The supplementary part of hands on activity 3 highlighted how the concept of circular doubly linked list can be implemented to design a simple music playlist. With the use of this function, a song will be stored in each node which will include the name of the song with links to the next and previous songs respectively. Furthermore, the circular nature of the doubly linked list makes sure that the playlist will loop from the last song to the first song making it a good choice for designing music player applications. In other words, this program implements various functionalities that are necessary for a playlist such as adding a song, deleting a song, moving to the next/previous song, displaying the currently playing song and playing all songs once using a menu driven approach. In addition to that, this model as we can see in the output section is heavily related to the old push-button phone or MP3. Overall, I enjoy doing this activity because I'm really interested in music while reinforcing what I have learned these past weeks in Data Structure concepts such as classes, objects, pointers, dynamic memory allocation, and data structure manipulation in C++.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance and Code Error Explanation

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The main idea of this work comes from me, and the AI only paves the way for me to reduce errors before compiling, and for explanation of errors, as well as to recommend concepts that might be helpful with this activity.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.